

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №2
по курсу «Программирование»
на тему «Оценка времени выполнения программ»

Выполнили:

студенты группы 24ВВВ4

Симаков А.С.

Орлов Р.А.

Принял:

к. т. н., доцент Юрова О.В.

к. э. н., доцент Акифьев И.В.

Пенза 2025

Цель работы:

Оценить время выполнения программы на языке Си или их частей, которые могут использоваться средствами, предоставляемые библиотекой **time.h**. Данная библиотека содержит описания типов и прототипы функций для работы с датой и временем.

Общие сведения.

Типы данных:

1. `clock_t` - возвращается функцией `clock()`. Обычно определён как `int` или `long int`.
2. `time_t` - возвращается функцией `time()`. Обычно определён как `int` или `long int`.
3. `struct tm` - нелинейное, дискретное календарное представление времени.

Основные функции:

1. `clock_t clock(void)` - возвращает время, измеряемое процессором в тактах от начала выполнения программы, или `-1`, если оно не известно. Пересчет этого времени в секунды выполняется по формуле:

`clock() / CLOCKS_PER_SEC`

где `CLOCKS_PER_SEC` – константа, определяющая количество тактов системных часов в секунду.

2. `time_t time(time_t *tp)`

Возвращает текущее календарное время или `-1`, если это время не известно. Если указатель `tp` не равен `NULL`, то возвращаемое значение записывается также и в `*tp`.

3. `double difftime(time_t time2, time_t time1)`

Возвращает разность `time2-time1`, выраженную в секундах.

Практическая часть

Задание1:

1. Вычислить порядок сложности программы (*O*-символику).
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
```

```
// Функция для измерения времени перемножения матриц заданного размера
double measure_time(int n) {
```

```

int i, j, r;

// Выделяем память под матрицы динамически
int** a = (int**)malloc(n * sizeof(int*));
int** b = (int**)malloc(n * sizeof(int*));
int** c = (int**)malloc(n * sizeof(int*));
for (i = 0; i < n; i++) {
    a[i] = (int*)malloc(n * sizeof(int));
    b[i] = (int*)malloc(n * sizeof(int));
    c[i] = (int*)malloc(n * sizeof(int));
}

// Заполнение матриц случайными числами
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        a[i][j] = rand() % 100 + 1;
        b[i][j] = rand() % 100 + 1;
    }
}

// Замер времени начала вычислений
clock_t start = clock();

// Перемножение матриц
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        c[i][j] = 0;
        for (r = 0; r < n; r++) {
            c[i][j] += a[i][r] * b[r][j];
        }
    }
}

// Замер времени окончания вычислений
clock_t end = clock();

// Освобождаем память
for (i = 0; i < n; i++) {
    free(a[i]);
    free(b[i]);
    free(c[i]);
}
free(a);
free(b);
free(c);

return (double)(end - start) / CLOCKS_PER_SEC;
}

int main(void) {
    setlocale(LC_ALL, "Russian"); // Установка русской локали

    srand(time(NULL));

    // Увеличили максимальные размеры матриц для тестирования
    int sizes[] = { 100, 200, 400, 1000, 2000, 4000, 10000 };
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    printf("=====\n");
    printf("АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ ПЕРЕМНОЖЕНИЯ МАТРИЦ\n");
    printf("=====\n\n");

    printf("1. ПОРЯДОК СЛОЖНОСТИ ПРОГРАММЫ (O-СИМВОЛИКА)\n");
    printf("Алгоритм перемножения матриц имеет три вложенных цикла,\n");
    printf("каждый из которых выполняется n раз.\n");

```

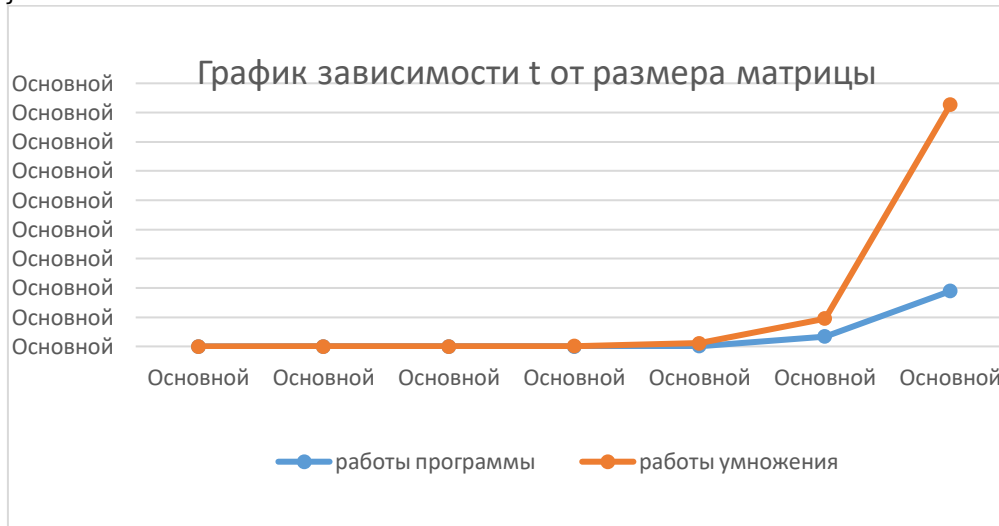
```

printf("Порядок сложности: O(n³)\n\n");

printf("2. РЕЗУЛЬТАТЫ ИЗМЕРЕНИЯ ВРЕМЕНИ ВЫПОЛНЕНИЯ\n");
printf("Размер матрицы | Время (сек) | Теоретическая оценка O(n³)\n");
printf("-----|-----|-----\n");

for (int i = 0; i < num_sizes; i++) {
    int n = sizes[i];
    double time_taken = measure_time(n);
    printf("%13d | %11.3f | O(%d³) = O('%d')\n", n, time_taken, n, n * n * n);
}
    getchar();
    return 0;
}

```



Результат программы:

```

=====
АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ ПЕРЕМНОЖЕНИЯ МАТРИЦ
=====

1. ПОРЯДОК СЛОЖНОСТИ ПРОГРАММЫ (O-СИМВОЛИКА)
Алгоритм перемножения матриц имеет три вложенных цикла,
каждый из которых выполняется n раз.
Порядок сложности: O(n³)

2. РЕЗУЛЬТАТЫ ИЗМЕРЕНИЯ ВРЕМЕНИ ВЫПОЛНЕНИЯ
Размер матрицы | Время (сек) | Теоретическая оценка O(n³)
-----|-----|-----
100 | 0,003 | O(100³) = O('d)
200 | 0,025 | O(200³) = O('d)
400 | 0,203 | O(400³) = O('d)
1000 | 4,403 | O(1000³) = O('d)
2000 | 51,325 | O(2000³) = O('d)
4000 | 1692,897 | O(4000³) = O('d)
10000 | 9452,847 | O(10000³) = O('d)

```

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.

3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
5. Оценить время работы стандартной функции qsort, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <locale.h>

// Объявления функций сортировки
void shell(int *items, int count);
void qs(int *items, int left, int right);

// Функция сравнения для qsort
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

// Функции заполнения массива
void fill_random(int *arr, int n) {
    for (int i = 0; i < n; i++) arr[i] = rand() % 10000;
}

void fill_ascending(int *arr, int n) {
    for (int i = 0; i < n; i++) arr[i] = i;
}

void fill_descending(int *arr, int n) {
    for (int i = 0; i < n; i++) arr[i] = n - i;
}

void fill_asc_desc(int *arr, int n) {
    int half = n / 2;
    for (int i = 0; i < half; i++) {
        arr[i] = i; // Первая половина возрастает
    }
    for (int i = half; i < n; i++) {
        arr[i] = n - (i - half); // Вторая половина убывает
    }
}

// Универсальная функция измерения времени
double measure_time(void (*sort_func)(int*, int), int *arr, int n) {
    int *arr_copy = (int *)malloc(n * sizeof(int));
    memcpy(arr_copy, arr, n * sizeof(int));

    clock_t start = clock();
    sort_func(arr_copy, n);
    clock_t end = clock();

    free(arr_copy);
    return (double)(end - start) / CLOCKS_PER_SEC;
}

double measure_time_qs(void (*sort_func)(int*, int, int), int *arr, int n) {
    int *arr_copy = (int *)malloc(n * sizeof(int));
    memcpy(arr_copy, arr, n * sizeof(int));
```

```

    clock_t start = clock();
    sort_func(arr_copy, 0, n - 1);
    clock_t end = clock();

    free(arr_copy);
    return (double)(end - start) / CLOCKS_PER_SEC;
}

double measure_time_std_qsort(int *arr, int n) {
    int *arr_copy = (int *)malloc(n * sizeof(int));
    memcpy(arr_copy, arr, n * sizeof(int));

    clock_t start = clock();
    qsort(arr_copy, n, sizeof(int), compare);
    clock_t end = clock();

    free(arr_copy);
    return (double)(end - start) / CLOCKS_PER_SEC;
}

// Реализации сортировок
void shell(int *items, int count) {
    int i, j, gap, k;
    int x, a[5] = {9, 5, 3, 2, 1};

    for (k = 0; k < 5; k++) {
        gap = a[k];
        for (i = gap; i < count; ++i) {
            x = items[i];
            for (j = i - gap; j >= 0 && x < items[j]; j = j - gap)
                items[j + gap] = items[j];
            items[j + gap] = x;
        }
    }
}

void qs(int *items, int left, int right) {
    int i = left, j = right;
    int x = items[(left + right) / 2], y;

    do {
        while (items[i] < x && i < right) i++;
        while (x < items[j] && j > left) j--;
        if (i <= j) {
            y = items[i];
            items[i] = items[j];
            items[j] = y;
            i++; j--;
        }
    } while (i <= j);

    if (left < j) qs(items, left, j);
    if (i < right) qs(items, i, right);
}

int main(void) {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    const int n = 10000;
    int *arr = (int *)malloc(n * sizeof(int));

    void (*fills[])(int*, int) = {fill_random, fill_ascending, fill_descending,
    fill_asc_desc};

```

```

const char *types[] = {"Случайные", "Возрастание", "Убывание", "Возр.+Убыв."};

printf("Тип данных      | Шелл (c) | QSort (c) | Std QSort (c)\n");
printf("-----|-----|-----|-----\n");

for (int i = 0; i < 4; i++) {
    fills[i](arr, n);

    double t_shell = measure_time(shell, arr, n);
    double t_qs = measure_time_qs(qs, arr, n);
    double t_std = measure_time_std_qsort(arr, n);

    printf("%-14s | %8.4f | %9.4f | %12.4f\n", types[i], t_shell, t_qs, t_std);
}
    getchar();
    free(arr);
    return 0;
}

```

Результат программы:

Тип данных	Шелл (c)	QSort (c)	Std QSort (c)
-----	-----	-----	-----
Случайные	0,0050	0,0010	0,0030
Возрастание	0,0000	0,0000	0,0020
Убывание	0,0100	0,0000	0,0020
Возр.+Убыв.	0,0020	0,0010	0,0010

Вывод:

Оценили время выполнения программы на языке Си или их частей, которые могут использоваться средствами, предоставляемые библиотекой **time.h**. Данная библиотека содержит описания типов и прототипы функций для работы с датой и временем.